

Guía de Componentes

MiniApi — .NET 10 Minimal API

Serilog · SQLite + Dapper · Health Checks · Swagger

1. Introducción

Este documento describe los cuatro componentes principales utilizados en el proyecto MiniApi: el sistema de logging con Serilog, la capa de persistencia con SQLite y Dapper, el sistema de Health Checks, y la documentación interactiva con Swagger/OpenAPI.

Cada sección explica el propósito del componente, los paquetes NuGet necesarios, y la configuración aplicada en el proyecto.

2. Logging — Serilog

2.1 ¿Qué es Serilog?

Serilog es una librería de logging estructurado para .NET. A diferencia del logging estándar de Microsoft (que emite texto plano), Serilog permite capturar propiedades con nombre que pueden filtrarse, enriquecerse y enviarse a múltiples destinos (llamados "sinks") de forma independiente.

2.2 Paquetes NuGet

Paquete	Versión	Descripción
Serilog	4.3.1	Núcleo del framework de logging
Serilog.AspNetCore	9.x	Integración con ASP.NET Core (UseSerilogRequestLogging)
Serilog.Sinks.Console	6.1.1	Escribe logs en la consola
Serilog.Sinks.File	7.0.0	Escribe logs en archivos con rotación diaria

2.3 Arquitectura de logging

El proyecto implementa una arquitectura de doble capa con dos destinos independientes:

Consola	Solo muestra eventos de nivel Error o superior. Mantiene la terminal limpia durante el desarrollo.
Archivo	Registra únicamente las requests HTTP procesadas por el middleware de Serilog. Excluye rutas /health y /swagger para evitar ruido. El archivo se rota diariamente en la carpeta logs/.

2.4 Configuración — LoggingExtensions.cs

La configuración se centraliza en el método de extensión AddAppLogging():

```
Log.Logger = new LoggerConfiguration()
    .MinimumLevel.Information()
    .MinimumLevel.Override("Microsoft", LogEventLevel.Warning)
    .MinimumLevel.Override("Microsoft.AspNetCore.Hosting.Diagnostics",
        LogEventLevel.Information)
    .Enrich.FromLogContext()

    // CONSOLA: solo errores
    .WriteTo.Logger(lc => lc
        .Filter.ByIncludingOnly(le => le.Level >= LogEventLevel.Error))
```

```

        .WriteTo.Console(outputTemplate: "[{Timestamp:HH:mm:ss} {Level:u3}]
{Message:lj}{NewLine}{Exception}")

// ARCHIVO: solo requests HTTP (sin /health ni /swagger)
        .WriteTo.Logger(lc => lc
            .Filter.ByIncludingOnly(le => {
                var esSerilogMiddleware = Matching
                    .FromSource("Serilog.AspNetCore.RequestLoggingMiddleware") (le);
                if (!esSerilogMiddleware) return false;
                if (le.Properties.TryGetValue("RequestPath", out var p) &&
                    p is ScalarValue s && s.Value is string path)
                    return !path.Contains("/health") && !path.Contains("/swagger");
                return true;
            })
            .WriteTo.File(
                path: "logs/audit.log",
                outputTemplate: "{Timestamp:yyyy-MM-dd HH:mm:ss} | {RequestMethod} |
{RequestPath} | {StatusCode}{NewLine}",
                rollingInterval: RollingInterval.Day))
        .CreateLogger();

builder.Host.UseSerilog();

```

2.5 Registro de requests en el pipeline

En `MiddlewareExtensions.cs` se configura el middleware que intercepta cada request HTTP y genera el evento de log:

```

app.UseSerilogRequestLogging(options =>
{
    options.GetLevel = (httpContext, _, ex) =>
        (ex != null) ? LogEventLevel.Error :
        (httpContext.Request.Path.StartsWithSegments("/health"))
            ? LogEventLevel.Verbose : LogEventLevel.Information;
});

```

Nota: Las requests a `/health` se registran como **Verbose** para no ensuciar el archivo de auditoría.

2.6 Niveles de log

Serilog (al igual que el sistema de logging de .NET) define 6 niveles de severidad en orden ascendente. Cada nivel filtra hacia arriba: configurar un mínimo de `Information` descarta `Verbose` y `Debug`, pero acepta `Warning`, `Error` y `Fatal`.

Nivel	Valor	Cuándo usarlo
Verbose	0	Trazas muy detalladas, solo para diagnóstico profundo. En este proyecto se usa para las requests a <code>/health</code> .
Debug	1	Información de depuración durante el desarrollo. Ej: valores de variables, flujo de ejecución, inicialización de componentes.

Nivel	Valor	Cuándo usarlo
Information	2	Eventos normales del sistema. Ej: requests HTTP recibidas, base de datos inicializada, aplicación iniciada. Es el nivel mínimo configurado en este proyecto.
Warning	3	Situaciones inesperadas pero no críticas. La aplicación sigue funcionando. Ej: configuración faltante con valor por defecto, reintentos de conexión.
Error	4	Errores que impiden completar una operación. La app sigue corriendo pero algo falló. Ej: error de conexión a la DB, excepción no manejada en un endpoint. Es el nivel mínimo de la consola en este proyecto.
Fatal	5	Error crítico que causa la caída de la aplicación. Ej: fallo al iniciar el servidor, corrupción de datos irrecuperable.

Configuración en este proyecto

El nivel mínimo global es Information, con overrides específicos para reducir el ruido de los namespaces de Microsoft:

```
.MinimumLevel.Information() // global
.MinimumLevel.Override("Microsoft", LogEventLevel.Warning) // menos ruido
.MinimumLevel.Override("Microsoft.AspNetCore.Hosting.Diagnostics",
LogEventLevel.Information)
```

Cómo emitir logs desde el código

```
// Inyectá ILogger<T> en tu clase
public class MiClase(ILogger<MiClase> logger)
{
    public void EjemploLogs()
    {
        logger.LogVerbose("Traza detallada");
        logger.LogDebug("Valor de variable: {Valor}", miVariable);
        logger.LogInformation("Operación completada para el item {Id}", id);
        logger.LogWarning("Configuración no encontrada, usando default");
        logger.LogError(ex, "Error al procesar el item {Id}", id);
        logger.LogCritical("Error fatal, cerrando aplicación");
    }
}
```

3. Base de Datos — SQLite + Dapper

3.1 ¿Qué son SQLite y Dapper?

SQLite es un motor de base de datos relacional embebido que almacena toda la información en un único archivo (.db). No requiere servidor ni instalación adicional, lo que lo hace ideal para proyectos de desarrollo y académicos.

Dapper es un micro-ORM (Object-Relational Mapper) que extiende IDbConnection con métodos de extensión para ejecutar consultas SQL y mapear los resultados a objetos C# de forma simple y eficiente.

3.2 Paquetes NuGet

Paquete	Versión	Descripción
Microsoft.Data.Sqlite	10.0.7	Driver oficial de Microsoft para SQLite en .NET
Dapper	2.1.72	Micro-ORM para mapeo SQL → objetos C#
SQLite	3.13.0	Librería nativa de SQLite

3.3 Cadena de conexión

La cadena de conexión se define en appsettings.json (o como fallback en el código):

```
// appsettings.json
{
  "ConnectionStrings": {
    "DefaultConnection": "Data Source=app.db"
  }
}

// Fallback en código
var connectionString = _config.GetConnectionString("DefaultConnection")
?? "Data Source=app.db";
```

El archivo app.db se crea automáticamente en la carpeta raíz del proyecto al iniciar la aplicación.

3.4 Inicialización de la base de datos — DatabaseInitializer.cs

Al arrancar la aplicación, el DatabaseInitializer crea la tabla si no existe:

```

public void Initialize()
{
    using var connection = new SqliteConnection(connectionString);
    connection.Open();

    connection.Execute("""
CREATE TABLE IF NOT EXISTS items (
    id            INTEGER PRIMARY KEY AUTOINCREMENT,
    name          TEXT      NOT NULL,
    description    TEXT,
    price         REAL      NOT NULL DEFAULT 0,
    stock         INTEGER NOT NULL DEFAULT 0,
    created_at    TEXT      NOT NULL DEFAULT (datetime('now')),
    updated_at    TEXT
);
""");
}

```

El método se llama desde Program.cs durante el arranque de la aplicación:

```

using (var scope = app.Services.CreateScope())
    scope.ServiceProvider
        .GetRequiredService<DatabaseInitializer>()
        .Initialize();

```

3.5 Repositorio — IRepository.cs

El repositorio encapsula todas las operaciones CRUD usando Dapper. Cada método abre una conexión, ejecuta la query y cierra la conexión automáticamente gracias al using:

```

// GET ALL
public async Task<IEnumerable<Item>> GetAllAsync()
{
    using var conn = CreateConnection();
    return await conn.QueryAsync<Item>("""
        SELECT id, name, description, price, stock,
               created_at AS CreatedAt, updated_at AS UpdatedAt
        FROM items ORDER BY id DESC
        """);
}

// CREATE
public async Task<Item> CreateAsync(CreateItemRequest request)
{
    using var conn = CreateConnection();
    var id = await conn.ExecuteScalarAsync<int>("""
        INSERT INTO items (name, description, price, stock)
        VALUES (@Name, @Description, @Price, @Stock);
        SELECT last_insert_rowid();
        """, request);
    return (await GetByIdAsync(id))!;
}

```

3.6 Ver el archivo de base de datos

El archivo app.db puede visualizarse sin instalar nada usando el servicio online:

URL	https://sqliteviewer.app
Instrucción	Arrastrá el archivo app.db desde la carpeta del proyecto al navegador
Ubicación	Raíz del proyecto → app.db

4. Health Checks

4.1 ¿Qué son los Health Checks?

Los Health Checks son endpoints especiales que permiten verificar el estado de la aplicación y sus dependencias. Son utilizados por herramientas de monitoreo, load balancers y sistemas de orquestación (como Kubernetes) para determinar si una instancia está sana.

4.2 Paquetes NuGet

Paquete	Versión	Descripción
AspNetCore.HealthChecks.UI	9.0.0	Dashboard web para visualizar el estado
AspNetCore.HealthChecks.UI.Client	9.0.0	Formatea la respuesta JSON del endpoint /health
AspNetCore.HealthChecks.UI.InMemory.Storage	9.0.0	Almacena el historial en memoria (sin base de datos extra)

4.3 Checks implementados

SqliteHealthCheck	Abre una conexión a la base de datos y ejecuta SELECT 1. Reporta Healthy si responde correctamente, Unhealthy si hay error de conexión.
ApiStatusCheck	Verifica que la API esté operativa. Retorna información de uptime, versión de .NET y timestamp de inicio.

4.4 Registro de servicios — ServicesExtensions.cs

```
services.AddHealthChecks()
    .AddCheck<SqliteHealthCheck>("sqlite-db", tags: ["database"])
    .AddCheck<ApiStatusCheck>("api-status", tags: ["api"]);

services.AddHealthChecksUI(setup =>
{
    setup.SetEvaluationTimeInSeconds(600); // evalúa cada 10 minutos
    setup.AddHealthCheckEndpoint("MiApi", "/health");
}).AddInMemoryStorage();
```


4.5 Configuración del pipeline — MiddlewareExtensions.cs

```
// Endpoint JSON con estado detallado
app.MapHealthChecks("/health", new HealthCheckOptions
{
    ResponseWriter = UIResponseWriter.WriteHealthCheckUIResponse
});

// Dashboard web
app.MapHealthChecksUI(setup => setup.UIPath = "/health-ui");
```

4.6 Endpoints disponibles

JSON de estado	https://localhost:7001/health
Dashboard UI	https://localhost:7001/health-ui

El dashboard se actualiza automáticamente cada 10 minutos (configurado con `SetEvaluationTimeInSeconds(600)`).

4.7 SqliteHealthCheck — código completo

```
public class SqliteHealthCheck : IHealthCheck
{
    private readonly IConfiguration _config;
    public SqliteHealthCheck(IConfiguration config) => _config = config;

    public async Task<HealthCheckResult> CheckHealthAsync(
        HealthCheckContext context,
        CancellationToken cancellationToken = default)
    {
        try
        {
            var connectionString = _config.GetConnectionString("DefaultConnection")
                ?? "Data Source=app.db";
            using var conn = new SqliteConnection(connectionString);
            await conn.OpenAsync(cancellationToken);
            await conn.ExecuteScalarAsync<int>("SELECT 1");
            return HealthCheckResult.Healthy("SELECT 1 ejecutado OK");
        }
        catch (Exception ex)
        {
            return HealthCheckResult.Unhealthy(
                description: "No se pudo conectar a SQLite",
                exception: ex);
        }
    }
}
```

5. Documentación — Swagger / OpenAPI

5.1 ¿Qué es Swagger?

Swagger (también conocido como OpenAPI) es un estándar para documentar APIs REST. Provee una interfaz web interactiva que permite ver todos los endpoints disponibles, sus parámetros, tipos de datos esperados, y ejecutar requests directamente desde el navegador sin necesidad de herramientas externas como Postman.

5.2 Paquetes NuGet

Paquete	Versión	Descripción
Swashbuckle.AspNetCore	10.1.7	Genera la interfaz web de Swagger UI
Swashbuckle.AspNetCore.Swagger	10.1.7	Genera el documento JSON OpenAPI
Microsoft.AspNetCore.OpenApi	10.0.6	Soporte nativo de OpenAPI en .NET

5.3 Registro de servicios — ServicesExtensions.cs

```
services.AddEndpointsApiExplorer(); // descubre los endpoints Minimal API
services.AddSwaggerGen();           // genera la especificación OpenAPI
```

5.4 Habilitación en el pipeline — Program.cs

```
if (app.Environment.IsDevelopment())
{
    app.UseSwagger(); // expone el JSON en /swagger/v1/swagger.json
    app.UseSwaggerUI(); // expone la UI en /swagger
}
```

Swagger solo se habilita en el entorno Development. En producción, el endpoint no está disponible por razones de seguridad.

5.5 Acceso

Swagger UI	https://localhost:7001/swagger
------------	---

JSON OpenAPI	https://localhost:7001/swagger/v1/swagger.json
Disponible en	Solo entorno Development (cuando se corre con F5 en Visual Studio)

6. Resumen — Todos los paquetes NuGet

Paquete	Versión	Descripción
<code>Serilog</code>	4.3.1	Núcleo del sistema de logging estructurado
<code>Serilog.AspNetCore</code>	9.x	Integración de Serilog con ASP.NET Core
<code>Serilog.Sinks.Console</code>	6.1.1	Sink de consola para Serilog
<code>Serilog.Sinks.File</code>	7.0.0	Sink de archivos con rotación diaria
<code>Microsoft.Data.Sqlite</code>	10.0.7	Driver oficial SQLite para .NET
<code>Dapper</code>	2.1.72	Micro-ORM para mapeo SQL → objetos
<code>SQLite</code>	3.13.0	Librería nativa de SQLite
<code>AspNetCore.HealthChecks.UI</code>	9.0.0	Dashboard web de Health Checks
<code>AspNetCore.HealthChecks.UI.Client</code>	9.0.0	Formateador JSON para /health
<code>AspNetCore.HealthChecks.UI.InMemory.Storage</code>	9.0.0	Almacenamiento en memoria del historial
<code>Swashbuckle.AspNetCore</code>	10.1.7	Swagger UI para documentación interactiva
<code>Swashbuckle.AspNetCore.Swagger</code>	10.1.7	Generador de especificación OpenAPI
<code>Microsoft.AspNetCore.OpenApi</code>	10.0.6	Soporte nativo OpenAPI en .NET 10